Analysis of Leet Code Problem Space

G Bhanu Prakash

K Aditya

B Hari Charan Goud

K Nithin

Y Suchethan Reddy

CS551 - Network Science

Indian Institute of Technology Bhilai

Abstract

Networks provide a powerful framework for analyzing complex real-world systems by representing entities as nodes and their relationships as connections. This approach has been widely used in domains such as social networks, biological systems, and information networks to uncover hidden patterns, identify influential components, and understand structural organization. Applying similar principles to algorithmic problem datasets can offer deeper insights into how concepts are interconnected and how knowledge is structured.

In this work, the LeetCode problem set is modeled as a network, where problems and their associated topic tags form a bipartite graph, along with its corresponding projections. Using network science techniques such as centrality measures, community detection, and co-occurrence analysis, the study explores relationships between problems and topics, identifies dominant concepts, and reveals underlying clusters representing algorithmic patterns.

The analysis shows that the problem space is highly structured, with a small number of core topics acting as hubs and connecting a large portion of the network. Furthermore, the network structure reflects problem difficulty and conceptual dependencies, enabling the identification of efficient learning strategies. A minimal subset of problems is also determined that covers the majority of concepts, highlighting redundancy within the dataset.

Overall, this work demonstrates how network-based modeling can provide a systematic and data-driven approach to understanding problem distributions and optimizing learning in algorithmic practice.

Contents

1 Introduction **1**

2 Data Collection and Preprocessing **3**

2.1 Data Collection 3

2.2 Dataset Description 3

2.3 Data Preprocessing 4

2.4 Data Preparation for Network Construction 4

2.5 Summary 5

3 Network Construction **6**

3.1 Problem–Tag Bipartite Network 6

3.1.1 Overview 6

3.1.2 Construction Method 6

3.1.3 Visualization of the Bipartite Network 6

3.1.4 Graph Properties 7

3.1.5 Tag Distribution 7

3.1.6 Problem Degree Analysis 8

3.1.7 Difficulty vs Degree Relationship 8

3.1.8 Tag Difficulty Profiles 9

3.1.9 Structural Insights 9

3.1.10 Optimization Insight (Set Cover) 10

3.1.11 Conclusion 10

3.2 Tag-Tag Co-occurrence Network 11

3.2.1 Frequent Tags (Degree Analysis) 12

3.2.2 Tag-Tag Co-occurrence Strength (Edge Weight Analysis) 12

3.2.3 Histogram of Tag Co-occurrence Frequencies 13
3.2.4 Betweenness Centrality Analysis 14
3.2.5 Community Detection (Modularity) 15
3.2.6 Tags per Community Distribution 16
3.2.7 Individual Community Analysis 16

Introduction

In recent years, network-based representations have become a powerful tool for analyzing complex systems across various domains such as social networks, biological systems, and information networks. By modeling entities as nodes and their relationships as edges, network science enables the discovery of hidden patterns, identification of influential components, and understanding of structural organization. Applying similar techniques to algorithmic problem datasets can provide valuable insights into how concepts are interconnected and how knowledge is structured.

Online coding platforms such as LeetCode host thousands of problems categorized by difficulty and associated topic tags. However, traditional approaches to problem-solving often involve solving problems sequentially or randomly, which can lead to redundancy and inefficient conceptual coverage. Many problems share similar underlying patterns, while some act as bridges connecting different domains. Without a structured approach, learners may spend significant effort on repetitive concepts while missing important connections between topics.

To address this limitation, this project models the LeetCode problem set as a network and analyzes it using tools from network science. The dataset is represented as a bipartite graph between problems and their associated tags, which is further projected into problem–problem and tag–tag networks. These representations enable the study of problem similarity, concept co-occurrence, and structural dependencies between topics.

Using various network analysis techniques such as centrality measures, community detection, and graph-based optimization, this work aims to identify key structural properties of the problem space. In particular, the analysis focuses on discovering central problems, understanding relationships between concepts, detecting clusters corresponding to algorithmic patterns, and determining a minimal subset of problems that achieves maximum conceptual coverage.

The insights derived from this study provide a data-driven perspective on problem selection

and learning strategies. By leveraging the structure of the network, learners can focus on high-impact problems and core concepts, thereby improving efficiency and reducing redundancy in preparation.

Data Collection and Preprocessing

2.1 Data Collection

The dataset used in this study is obtained from the LeetCode problem set, which contains a large collection of algorithmic problems categorized by difficulty and associated topic tags. Since the LeetCode platform dynamically renders content, automated data extraction is performed using Selenium, a browser automation tool.

The data collection process involves the following steps:

1. Load the LeetCode problem set webpage using Selenium.
2. Dynamically scroll through the page to ensure that all problems are fully rendered.
3. Extract basic information such as problem ID, title, and difficulty level.
4. Visit individual problem pages to retrieve associated topic tags.
5. Store the collected data in a structured format (JSON).
6. Convert the JSON data into CSV format for further analysis.

This automated approach ensures comprehensive coverage of the problem set and enables efficient extraction of relevant attributes required for network modeling.

2.2 Dataset Description

Each problem in the dataset is represented using the following attributes:

- **Problem ID:** Unique identifier of the problem
- **Title:** Name of the problem

- **Difficulty:** Categorized as Easy, Medium, or Hard
- **Tags:** List of associated topic tags

An example representation of a problem entry is shown below:

```
{
  "number": "1",
  "title": "Two Sum",
  "difficulty": "Easy",
  "tags": ["Array", "Hash Table"]
}
```

The final dataset consists of thousands of problems and a diverse set of tags, forming the basis for constructing the network representations.

2.3 Data Preprocessing

Before constructing the network, the dataset undergoes preprocessing to ensure consistency and usability.

The following preprocessing steps are applied:

- **Data Cleaning:** Removal of missing or incomplete entries, if any.
- **Tag Normalization:** Standardizing tag names to avoid inconsistencies (e.g., handling variations in naming conventions).
- **Duplicate Removal:** Ensuring that each problem is uniquely represented.
- **Format Conversion:** Transforming the dataset into CSV format suitable for analysis using Python libraries.
- **Tag Parsing:** Converting tag strings into structured lists for easy processing.

2.4 Data Preparation for Network Construction

To enable graph-based analysis, the preprocessed dataset is transformed into formats suitable for different network representations:

- **Bipartite Graph Input:** Problem–tag pairs are extracted to form edges between problems and tags.
- **Problem Projection:** Problem–problem relationships are generated based on shared tags.
- **Tag Projection:** Tag–tag relationships are constructed based on co-occurrence within problems.

These transformations allow the dataset to be analyzed from multiple perspectives, enabling deeper insights into problem similarity, concept relationships, and structural properties of the problem space.

2.5 Summary

The data collection and preprocessing pipeline ensures that the LeetCode problem set is transformed into a structured and analyzable format. By leveraging automated scraping and systematic preprocessing, the dataset becomes suitable for network construction and subsequent graph-based analysis. This forms the foundation for all further investigations carried out in this study.

Network Construction

3.1 Problem–Tag Bipartite Network

3.1.1 Overview

The problem–tag network is modeled as a bipartite graph consisting of problems and their associated tags. The dataset contains 3,774 problems, 72 tags, and 11,761 edges, resulting in a sparse graph structure.

In this representation:

- One set of nodes represents problems
- The other set represents tags
- An edge exists if a problem is associated with a tag

3.1.2 Construction Method

The bipartite graph is constructed by iterating through the dataset and creating edges between problems and their corresponding tags. For each problem, edges are added to all associated tags, forming a mapping between problems and concepts.

3.1.3 Visualization of the Bipartite Network

The constructed problem–tag bipartite network is visualized using graph visualization tools. The figure below illustrates the structure of the network, where nodes represent problems and tags, and edges represent their associations.

Figure 3.1: Visualization of the Problem–Tag Bipartite Network

3.1.4 Graph Properties

- The graph is bipartite, with no edges within the same node set
- The degree of a problem node represents the number of associated tags
- The degree of a tag node represents the number of problems containing that tag

3.1.5 Tag Distribution

The distribution of tags in the problem–tag network exhibits a highly skewed, power-law-like behavior. A small number of tags dominate the network, while the majority appear infrequently.

For instance, the **Array** tag alone is associated with 2,127 problems, whereas several tags such as *Biconnected Component* and *Strongly Connected Component* appear in only one or two problems.

This results in a clear distinction between:

- **Hub tags**, which are widely used and occur frequently across problems
- **Niche tags**, which are rare and represent specialized topics

Insight: A small set of core tags, including Array, String, and Hash Table, covers a large portion of the problem space, while the long tail represents advanced or domain-specific knowledge.

3.1.6 Problem Degree Analysis

The number of tags associated with each problem, referred to as the degree, provides insight into the conceptual complexity of problems.

The average number of tags per problem is 3.1, with the distribution peaking at 3 tags. Approximately 14.8% of problems have only a single tag, indicating single-concept problems. In contrast, only 29 problems have 8 or more tags, representing highly complex, multi-concept problems.

The distribution is right-skewed, suggesting that most problems involve multiple concepts rather than being isolated to a single topic.

Insight:

- Low-degree problems correspond to simpler, single-concept problems
- High-degree problems represent complex problems requiring integration of multiple concepts

Figure 3.2: Your caption here

3.1.7 Difficulty vs Degree Relationship

A strong relationship is observed between problem difficulty and the number of associated tags. On average, Easy problems have 2.17 tags, Medium problems have 3.14 tags, and Hard problems have 3.69 tags. The median values also increase consistently from 2 to 3 to 4.

Insight: The degree of a problem node acts as a structural proxy for its difficulty. Harder problems tend to involve a broader range of concepts and occupy higher-degree positions in the network.

3.1.8 Tag Difficulty Profiles

Tags can be further analyzed based on their distribution across difficulty levels. Certain tags are predominantly associated with harder problems, while others are more common in easier ones.

Hard-Dominant Tags

Tags such as Segment Tree, Binary Indexed Tree, Bitmask, and Dynamic Programming are primarily associated with Medium and Hard problems. These tags rarely appear in Easy problems and represent deeper, more complex algorithmic concepts.

Easy-Friendly Tags

Tags such as Simulation, Database, Counting, and Two Pointers are more frequently observed in Easy problems and serve as entry points for beginners.

A notable exception is **Dynamic Programming**, which exhibits a bimodal distribution. It appears almost exclusively in Medium and Hard problems, with very few Easy instances, making it a key transition topic in the learning process.

3.1.9 Structural Insights

The bipartite structure of the network leads to several important observations. Tags act as central hubs connecting multiple problems, while problems serve as connectors between different concepts.

The network exhibits small-world characteristics, with a diameter of approximately 7, indicating that most nodes are reachable within a few steps. Additionally, clustering is minimal due to the bipartite nature of the graph.

It is also observed that certain domains, such as database-related problems, form isolated components. This suggests that these problems are structurally separate from the main algorithmic problem space.

3.1.10 Optimization Insight (Set Cover)

To evaluate redundancy in the dataset, a greedy set cover approach is applied to identify a minimal subset of problems that covers all tags.

The results show that only 26 problems are sufficient to cover all 72 tags. The first few selected problems contribute significantly to coverage, while subsequent problems provide diminishing returns.

Insight: The problem set is highly redundant, and a small subset of carefully selected problems can provide comprehensive conceptual coverage.

3.1.11 Conclusion

The analysis of the problem–tag network reveals that the problem space is structured around a few dominant tags, with a strong relationship between conceptual breadth and problem difficulty. The presence of redundancy and identifiable patterns enables the design of efficient learning strategies based on network structure.

Overall, the graph provides a clear representation of how problems, concepts, and difficulty levels are interconnected within the dataset.

3.2 Tag-Tag Co-occurrence Network

The constructed network represents relationships between tags based on their co-occurrence across problems. Each node corresponds to a unique tag, and an edge is formed between two tags if they appear together in at least one problem. The weight of an edge indicates the number of problems in which the corresponding pair of tags co-occur.

This network captures the structural relationships between different problem-solving concepts and provides a data-driven view of how topics interact in practice.

The network was constructed using extracted problem-tag data and visualized in Gephi. The final network consists of:

- Number of nodes (tags): **N**
- Number of edges (co-occurrences): **E**

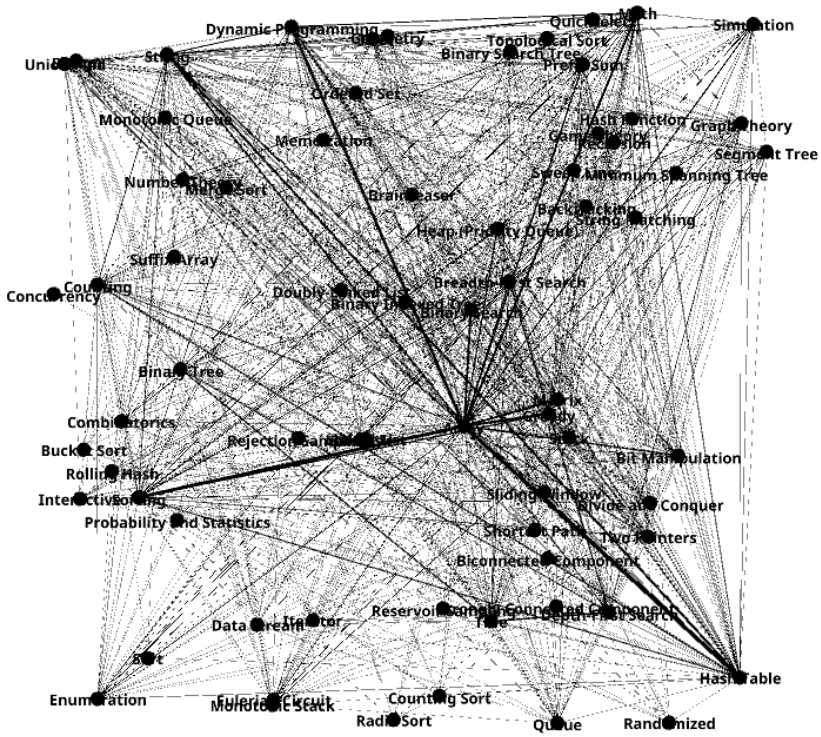

Figure 3.3: Tag-Tag Co-occurrence Network constructed in Gephi

3.2.1 Frequent Tags (Degree Analysis)

To identify the most frequently occurring and widely connected tags, we used **degree centrality**. The degree of a node represents the number of unique tags it co-occurs with, which serves as a proxy for how broadly a concept is used across problems.

In Gephi:

- Degree was computed using the statistics panel
- Node sizes were scaled according to degree values
- Additional coloring was applied to enhance visual distinction

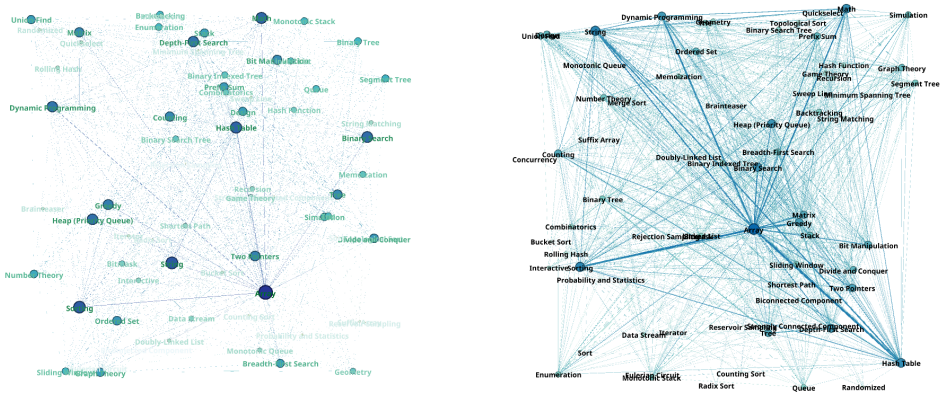

Figure 3.4: Visualization of tag frequency using degree centrality

Insight: Tags with high degree act as central concepts that connect multiple topics. These represent foundational techniques that are widely applicable across different problem types.

3.2.2 Tag-Tag Co-occurrence Strength (Edge Weight Analysis)

To analyze the strength of relationships between tags, we used **edge weights**, where each weight represents how frequently two tags appear together in problems.

In Gephi:

- Edge weights were used as a ranking attribute
- Edge thickness was scaled proportionally to weight

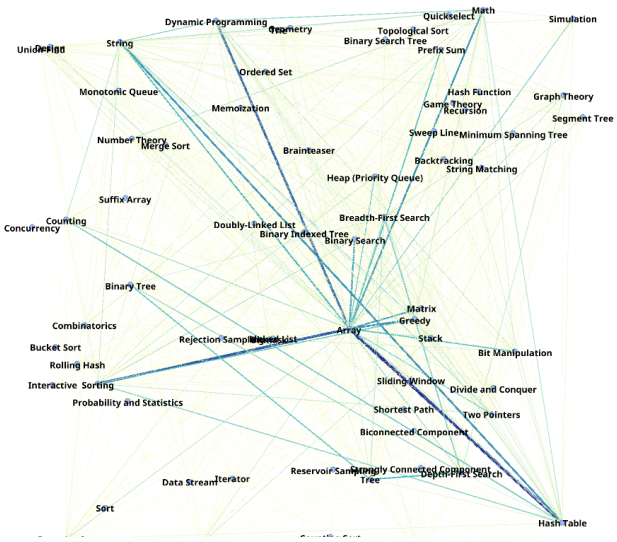

Figure 3.5: Edge weights representing tag co-occurrence frequency

Insight: Strong edges indicate pairs of concepts that frequently co-occur, suggesting strong conceptual relationships and commonly combined techniques in problem-solving.

3.2.3 Histogram of Tag Co-occurrence Frequencies

To better understand the distribution of co-occurrence strengths, a histogram of edge weights was generated.

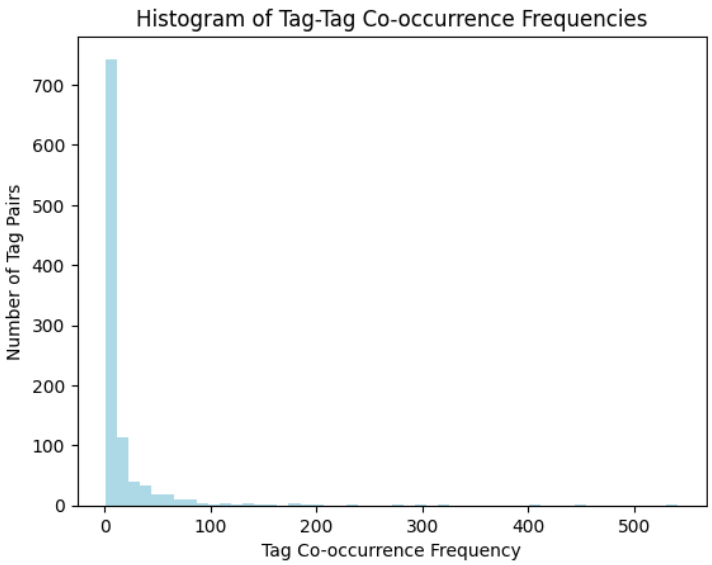

Figure 3.6: Histogram of tag co-occurrence frequencies

Insight: The histogram reveals a skewed distribution where most tag pairs occur infrequently,

while a small number of pairs occur very frequently. This indicates that while many topic combinations exist, only a few dominate in practice.

3.2.4 Betweenness Centrality Analysis

To identify tags that serve as connectors between different groups of topics, we computed **betweenness centrality**. This measure quantifies how often a node lies on the shortest paths between other nodes.

In Gephi:

- Betweenness was computed using the Network Diameter function
- Node sizes were scaled based on betweenness values

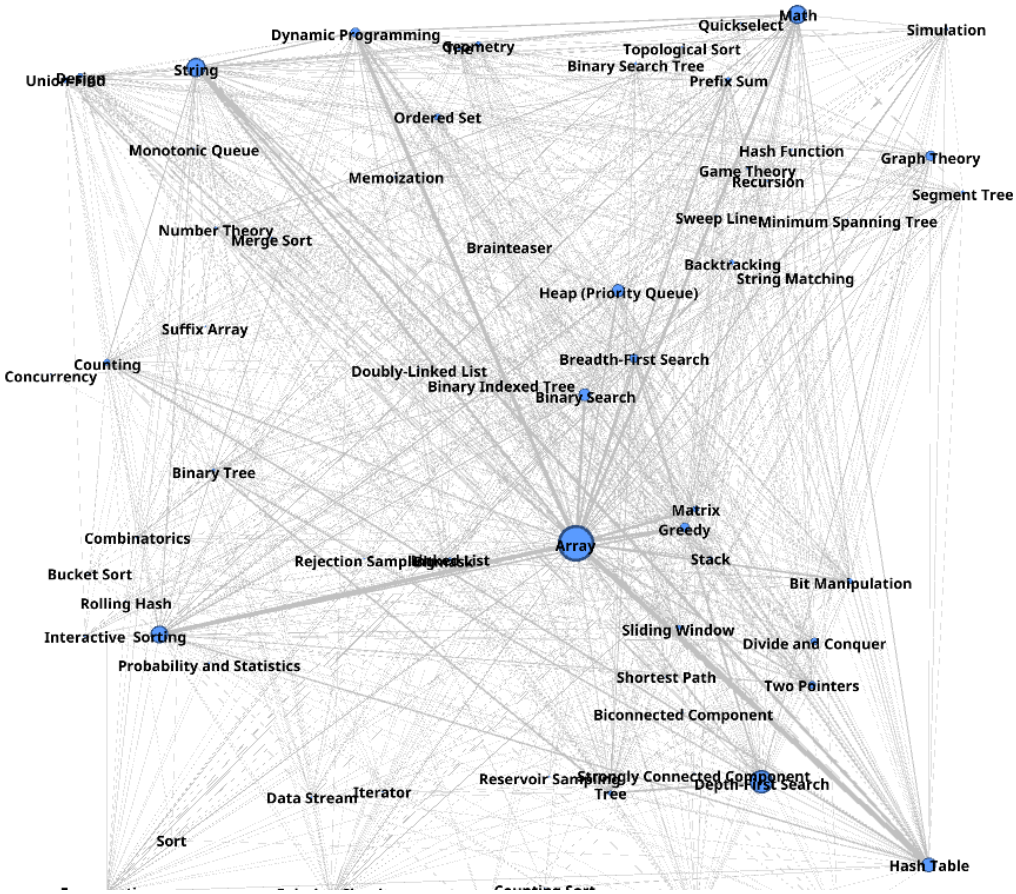

Figure 3.7: Betweenness centrality of tags

Insight: Tags with high betweenness act as bridges between different clusters, indicating concepts that connect multiple domains and enable transitions between different problem-solving

techniques.

3.2.5 Community Detection (Modularity)

To uncover structural groupings in the network, we applied **Louvain modularity-based community detection**. This method partitions the network into clusters such that nodes within a cluster are more densely connected to each other than to nodes outside the cluster.

In Gephi:

- Modularity was computed using the statistics panel
- Nodes were colored according to modularity class

A total of **5 communities** were identified.

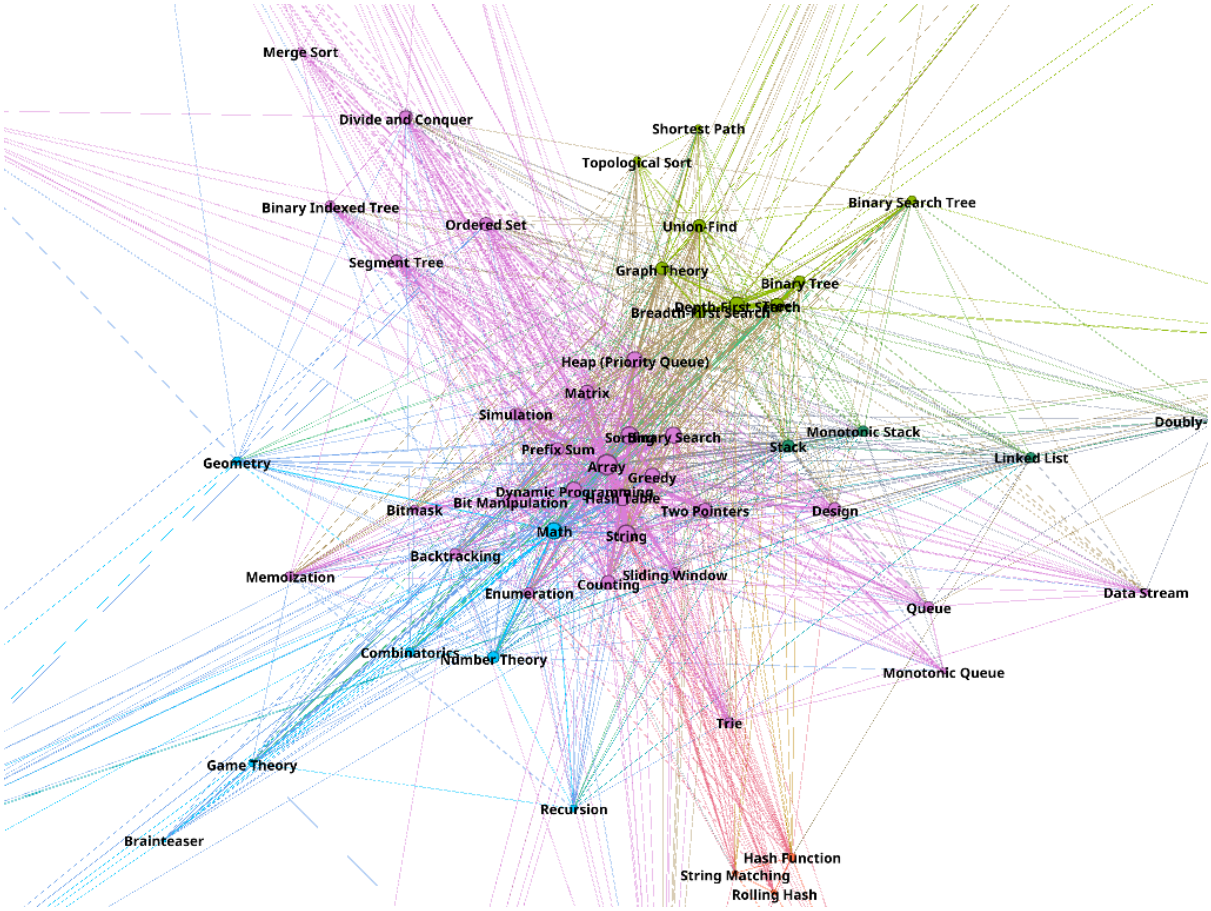

Figure 3.8: Communities detected in the tag network

Insight: Each community represents a group of closely related concepts, indicating that the problem space naturally organizes into distinct thematic clusters.

3.2.6 Tags per Community Distribution

The distribution of tags across communities was analyzed to understand the relative size of each cluster.

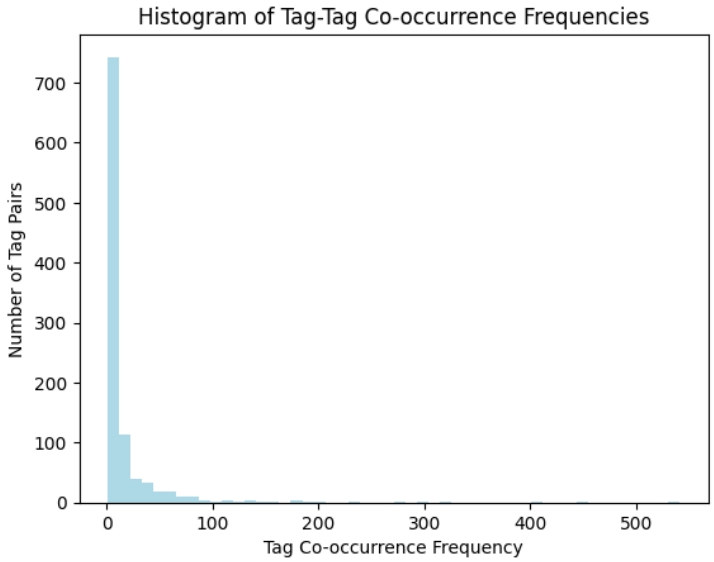

Figure 3.9: Distribution of tags across communities

Insight: Larger communities correspond to dominant and widely used topic groups, while smaller communities represent niche or specialized areas.

3.2.7 Individual Community Analysis

Each community was analyzed individually to understand its conceptual composition.

Community 0

Rolling Hash, Suffix Array, String Matching, Hash Function

Insight: This community is centered around string processing and hashing techniques, indicating problems related to pattern matching and text algorithms.

Community 1

Backtracking, Dynamic Programming, Array, Binary Search

Insight: This represents a core problem-solving cluster combining fundamental data structures with algorithmic paradigms, forming a major portion of typical problems.

Community 2

Recursion, Math, Probability, Combinatorics

Insight: This cluster corresponds to mathematically intensive problems, often involving counting, probability, and recursive formulations.

Community 3

Union-Find, Topological Sort, BFS, DFS, MST, BST

Insight: This community captures graph-based algorithms and tree structures, representing connectivity, traversal, and optimization problems.

Community 4

Linked List, Doubly Linked List, Stack, Monotonic Stack

Insight: This cluster focuses on linear data structures and stack-based techniques, often used in sequential processing problems.

Cross-Community Observations

While each community represents a distinct conceptual domain, there exist important relationships across communities. Certain tags act as bridges between clusters, indicating that real-world problems often require combining multiple paradigms. For example, dynamic programming concepts may interact with graph-based techniques, and mathematical reasoning may complement combinatorial or recursive approaches.

This interconnectivity suggests that the problem space is not strictly partitioned, but rather organized into overlapping conceptual regions.

Structural Interpretation of the Network

The overall structure of the tag-tag network reveals several important characteristics:

- The presence of dense communities indicates strong thematic grouping of concepts.

- High-degree nodes represent widely applicable techniques that appear across multiple domains.
- High-weight edges highlight frequently co-occurring concept pairs, forming common problem patterns.
- Bridge nodes (high betweenness) connect otherwise distinct clusters, enabling transitions between topics.

This demonstrates that the problem space is both *modular* and *interconnected*, which is a typical property of complex networks.

Implications for Problem Solving

The observed network structure has direct implications for learning and problem-solving strategies:

- Core communities (such as those containing arrays, dynamic programming, and graph algorithms) should be prioritized, as they cover a large portion of problems.
- Strongly connected tag pairs indicate commonly used combinations of techniques, suggesting that these concepts should be studied together.
- Smaller communities represent specialized areas, which can be approached after mastering foundational topics.
- Bridge concepts are particularly important, as they enable solving problems that span multiple domains.

Summary of Findings

From the network-based analysis, we observe that:

- The tag space is structured into a limited number of meaningful communities.
- A small number of tags act as central hubs in the network.
- Most tag combinations are rare, while a few combinations dominate problem construction.
- The organization of tags reflects practical problem-solving patterns rather than arbitrary categorization.

Overall Insight: The tag-tag network provides a structured representation of the problem space, revealing how different algorithmic concepts are organized, interconnected, and utilized in practice. This enables a deeper understanding of problem composition beyond individual topics.

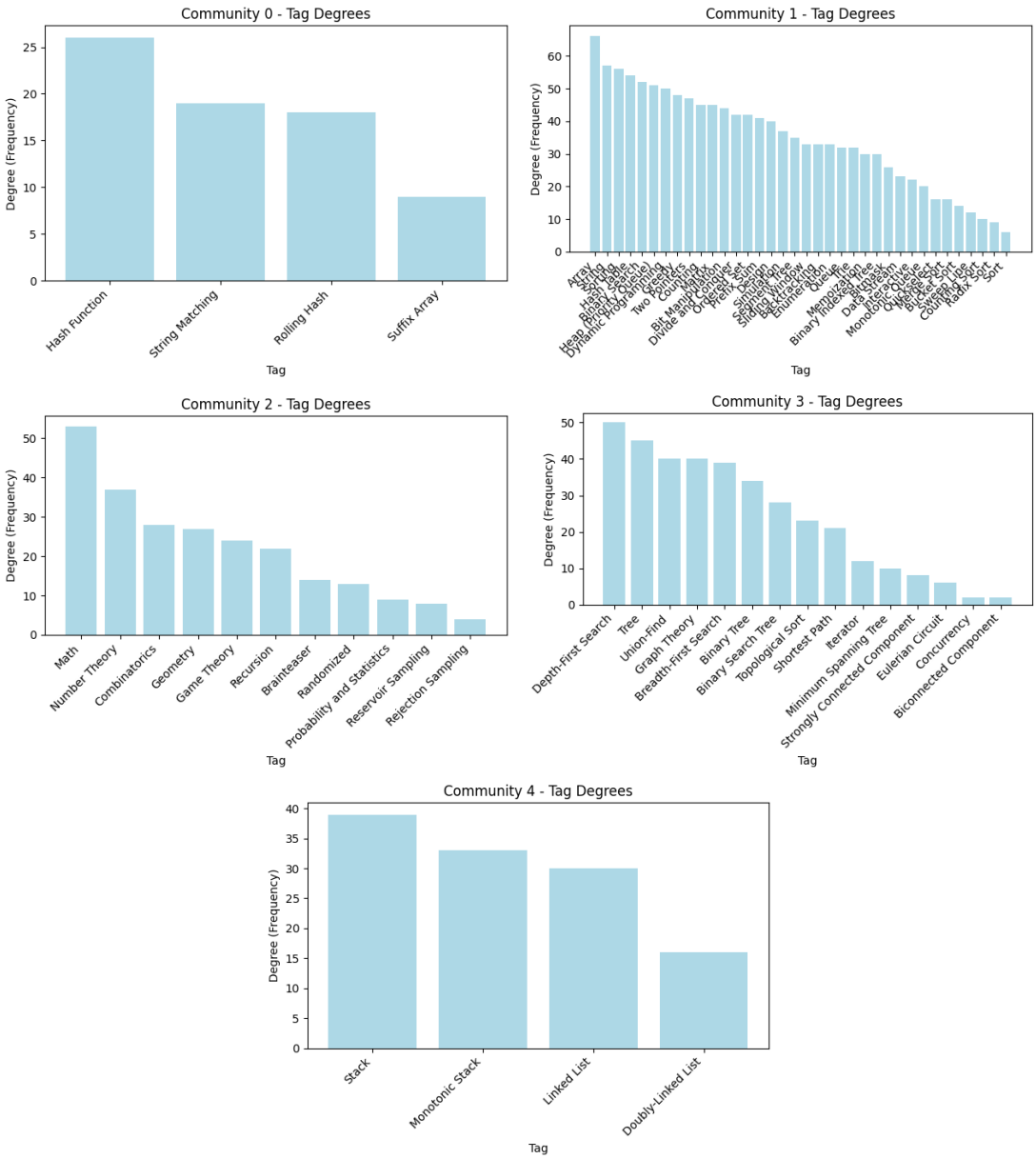

Figure 3.10: Histograms for individual communities

Insight: The decomposition of the network into these communities highlights that the problem space is not random but structured into well-defined conceptual domains.